

claude-windows / ac48091f-a4f4-4268-b968-75205d9a50ff

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ac48091f-a4f4-4268-b968-75205d9a50ff\subagents\workflows\wf_b8ec0894-3cb\agent-ad5238dc1c4667f62.jsonl`  
- SHA-256: `0d60c2731ac73ce96687d4046dc1d0e7d38d3d07b79cf2070865fe6ecc55b1a8`  
- Source modified: `2026-06-21T19:15:07+00:00`  
- Imported at: `2026-07-08T15:59:55+00:00`  
- project: `wf_b8ec0894-3cb`  
- session_id: `ac48091f-a4f4-4268-b968-75205d9a50ff`
```

Transcript

1. user (2026-06-21T19:12:53.044Z)

You are verifying ONE claim in khelsutra's deploy runbook (deploy/DEPLOY_ALPHA.md) against the REAL code the owner will deploy.

CRITICAL repo rules:

```
- Engine repo is at /c/Users/avidu/Projects/badminton-highlight-indexer. Its working tree is checked out on a STALE concurrent-session branch (12 commits behind master). DO NOT read the working tree for engine claims and DO NOT checkout/commit/modify anything there ? it is a shared checkout owned by another session.  
- To read what the owner actually deploys, read ENGINE origin/master ONLY, via:  
  git -C /c/Users/avidu/Projects/badminton-highlight-indexer grep -n "PATTERN" origin/master  
-- 'PATHGLOB'          (search master's tree)  
  git -C /c/Users/avidu/Projects/badminton-highlight-indexer show origin/master:PATH  
(read a master file)  
  git -C /c/Users/avidu/Projects/badminton-highlight-indexer log origin/master --oneline -i  
--grep="..."        (history)  
  (I already ran 'git fetch' so origin/master is current. Never fetch/checkout.)  
- khelsutra repo is at /c/Users/avidu/Projects/khelsutra, working tree is on clean master ? you MAY use Read/Grep/Glob directly there.
```

Be adversarial and literal: confirm the claim is TRUE for origin/master, with file:line evidence. If the doc says X but the code says Y, that is a 'discrepancy' ? capture it precisely. If something the owner must DO (beyond the doc's text) is implied, put it in owner_action_note. Verdict 'confirmed' only with concrete evidence; 'partial' if mostly true with a caveat; 'refuted' if wrong; 'unknown' if you genuinely could not determine it.

CLAIM (\$2.2/\$4.5): the engine locks CORS to the SPA origin via env `RALLY\_CORS\_ALLOW\_ORIGINS` (no code change, env-driven), with allow_credentials=false, and answers the cross-origin OPTIONS preflight. ALSO the doc says "Create/edit config.json on the box" (\$2.2).
On origin/master verify: (a) RALLY_CORS_ALLOW_ORIGINS is read and applied to CORS middleware; allow_credentials is false; (b) GOTCHA CHECK: packaging PR #293 added a resolved app-data HOME (RALLY_APP_HOME) for config/.env/DB/output. So WHERE does the engine actually look for config.json ? CWD, or under RALLY_APP_HOME, or an env-pointed path? The doc tells the owner to "create/edit config.json on the box" without saying where ? determine the real discovery path and whether the owner could put config.json somewhere the engine won't read. Report the exact resolution order.

2. assistant (2026-06-21T19:12:56.079Z)

I'll verify this claim about CORS configuration and config.json discovery on the engine's origin/master. Let me start by reading the deploy runbook and then investigating the engine code.

3. user (2026-06-21T19:12:58.628Z)

```

20-
21-- **No inbound ports.** `cloudflared` dials out; the engine stays bound to `127.0.0.1:8000`.
The box's home/public IP is never exposed.
22-- **Auth at the edge.** Cloudflare Access gates both hostnames; the engine *also* verifies
the `Cf-Access` JWT in-app (defence in depth ? `security.edge_auth_enabled`), so a direct hit to
the tunnel can't spoof identity.
23- **Cross-origin, on purpose.** `app.` (SPA) and `api.` (engine) are different origins ? the
SPA is built with `VITE_API_BASE=https://api.khelsutra.guru/api` (PR #64) and the engine locks
CORS to the SPA origin via `RALLY_CORS_ALLOW_ORIGINS` (no engine code change ? it's already env-
driven).
24-
25-> **Alternative (single-origin):** if you'd rather run one box that serves the SPA and
the API behind one origin (Caddy reverse-proxy, no Pages, no cross-origin), see
`LOCAL_APPLIANCE_ARCHITECTURE.md` D1. This runbook ships the split topology you chose
(stable Pages-hosted page + a thin API tunnel).
26-
--
44-```
45-
46-### 2.2 The safe-exposed config
47:Create / edit `config.json` on the box:
48-```jsonc
49-{
50-  "security": {
51-
52-
53-
54-
55-
56-
57-
58-
59-And export the env (a `.env` or your shell):
60-```bash
61-export GEMINI_API_KEY=""
62:export RALLY_CORS_ALLOW_ORIGINS="https://app.khelsutra.guru" # lock CORS to the SPA origin
63-```
64-
65-> The engine refuses to start unsafe. The boot posture (engine PR #294) fails fast if
`edge_auth_enabled` is on but the `allowed_video_root` jail or `cf_team_domain`/`cf_access_aud`
is missing, and warns if `PyJWT` isn't installed. So a half-configured exposure can't
silently come up ? fix what it prints and restart.
--
93-1. Zero Trust ? Access ? Applications ? Add ? Self-hosted.
94-2. Application domain: add both `app.khelsutra.guru` and `api.khelsutra.guru` to
the same application (one app ? one shared session cookie, so a tester signs in once and both
the SPA and its API calls are authorized).
95-3. Policy: Action Allow, rule Emails ? your alpha testers' addresses (free ? 50 users
? fits 10?25 testers). Use one-time-PIN (email OTP) so testers need no Cloudflare account.
96-4. Copy the application AUD tag ? paste into the engine's `config.json`
`security.cf_access_aud` (§2.2), and set `cf_team_domain` to `https://<your-
team>.cloudflareaccess.com`. Restart the engine.
97-5. CORS preflight ?: the browser sends an unauthenticated `OPTIONS` preflight to `api.`
for the cross-origin POSTs. In the Access application's Settings ? enable Bypass options
requests to CORS preflight? (or add an explicit OPTIONS bypass), else the preflight is
redirected to the login page and every API call fails. The engine's CORS middleware
(`allow_credentials=false`, no cookies) then answers the preflight.
98-
99-
100-

```

4. user (2026-06-21T19:12:59.744Z)

```

origin/master:backend/config/models.py:306: # (CORS allow-origins is set via the
RALLY_CORS_ALLOW_ORIGINS env var ? read at import in
origin/master:backend/main.py:66: """M9: CORS allow-origins from the
`RALLY_CORS_ALLOW_ORIGINS` env var (comma-separated),
origin/master:backend/main.py:70: raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or
"").strip()
origin/master:docs/COMPUTE_DECOUPLED_SERVING/README.md:166:- 2026-06-21 ? M9-auth code
landed* ([#290](https://github.com/Khelsutra/badminton-highlight-indexer/pull/290)):
`backend/api/auth.py` ? Cloudflare Access Cf-Access JWT verify (RS256: sig+exp+iss+aud

```

```

against the team JWKS; lazy `PyJWT[crypto]`; injectable JWKS ? fully-offline tests) +
`resolve_tenant` ? tags `create_job(owner_id=?)`;
`SecurityConfig.{edge_auth_enabled=False,cf_team_domain,cf_access_aud}` + CORS via
`RALLY_CORS_ALLOW_ORIGINS`. **Default-OFF.** Security review (`wf_9b4c71d0`) found NO bypass;
locked alg=none + HS256-confusion rejection. **? M9 ? ? the consent cols + ToS/DPA are
owner/counsel; the consent migration is now v7 (M8 took v6).**
origin/master:tests/test_edge_auth.py:205:      """CORS origins come from RALLY_CORS_ALLOW_ORIGINS
(import-safe, no cwd config I/O); default *."""
origin/master:tests/test_edge_auth.py:208:      assert
m._cors_origins_from_env({"RALLY_CORS_ALLOW_ORIGINS": ""}) == ["*"]
origin/master:tests/test_edge_auth.py:210:      {"RALLY_CORS_ALLOW_ORIGINS":
"https://a.example, https://b.example"}

```

5. assistant (2026-06-21T19:13:02.962Z)

Let me read the key engine files. The grep already revealed something important:
`test\_edge\_auth.py:205` says CORS origins come from the env "import-safe, no cwd config I/O".
Let me dig into the actual code and the config.json discovery path.

6. user (2026-06-21T19:13:05.109Z)

```

    load_config as load_app_config,
    AppConfig,
    StitchingConfig,
    enforce_startup_checks,
    StartupCheckError,
)
from backend.results import Failure # standardized error/result contract
from backend.reporting import emit_indexer_report
from backend import storage # M2: URI-aware input acquisition (local = identity passthrough)
from backend import billing # M8: per-job cost ledger (USD internal / INR display)
from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-OFF)

app = FastAPI(title="Sports Highlight Indexer API")

def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
    """M9: CORS allow-origins from the `RALLY_CORS_ALLOW_ORIGINS` env var (comma-separated),
    default `["*"]`. Read from the ENV (not a cwd `config.json`) so importing
    ``backend.main``
    does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to its
    origin."""
    src = os.environ if env is None else env
    raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
    origins = [o.strip() for o in raw.split(",") if o.strip()]
    return origins or ["*"]

```

CORS for the web UI. `allow_origins=*` with `allow_credentials=True` is rejected by browser. `fetch spec` + is unsafe to carry into multi-tenant; this tool uses no cookies, so credentials are not sent off. The origin list is configurable via the env var above (default `["*"]`) so a hosted deployment can lock it to its origin; the middleware is fixed at startup.

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=_cors_origins_from_env(),
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)

```

```

def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:

```

```

    """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A
malicious
    web page can point a domain it controls at 127.0.0.1 and script requests to the local
server;
    the browser sends ``Host: attacker.com``, which this rejects (the server only answers
loopback
    Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the default
keeps
    Starlette's TestClient host ``testserver`` so the in-process suite isn't broken. Read from
the
    ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a
routable/edge
    deploy sets its public host (or ``*`` when fronted by an auth gate that validates the
host)."""
    src = os.environ if env is None else env
    raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
    hosts = [h.strip() for h in raw.split(",") if h.strip()]
    # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's
TrustedHostMiddleware
    # matches on host.split(":")[0], which mangles a bracketed IPv6 Host ("[:1]:port"), so IPv6
    # loopback is NOT supported by this guard ? access the appliance via 127.0.0.1/localhost.
    return hosts or ["localhost", "127.0.0.1", "testserver"]

```

DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers a (a remote page can't read responses cross-origin AND its Host header is rejected here off so an unexpected ``www.`` host is rejected rather than 307-redirected.

```

app.add_middleware(
    TrustedHostMiddleware,
    allowed_hosts=_allowed_hosts_from_env(),
    www_redirect=False,
)

def _resolve_bind_host(config: Optional[AppConfig] = None, cli_host: Optional[str] = None) ->
str:
    """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env >
``0.0.0.0``
    when an edge-auth profile is active (``security.edge_auth_enabled`` ? i.e. there IS an auth
gate
    in front) > ``127.0.0.1`` (default: localhost-only, today's behaviour). Binding a routable
interface with no auth gate is unsafe, hence the localhost default; widen it consciously."""
    if cli_host and cli_host.strip():
        return cli_host.strip()
    env = os.environ.get("RALLY_BIND_HOST")
    if env and env.strip():
        return env.strip()
    sec = getattr(config, "security", None)
    if getattr(sec, "edge_auth_enabled", False):
        return "0.0.0.0" # behind a Cloudflare-Access-style edge gate (M9) ? bind all
interfaces
    return "127.0.0.1"

```

--- P2: the friendly `indexer --app` launcher (cross-platform, no native component ? O1

```

def _port_is_free(host: str, port: int) -> bool:
    """True if (host, port) can be bound right now (best-effort; a TOCTOU race is acceptable ?
uvicorn surfaces a real bind failure loudly)."""
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        try:
            s.bind((host, port))

```

```
        return True
    except OSError:
```

7. user (2026-06-21T19:13:06.239Z)

```
origin/master:backend/main.py:32:from fastapi.middleware.cors import CORSMiddleware
origin/master:backend/main.py:65:def _cors_origins_from_env(env: Optional[Dict[str, str]] =
None) -> List[str]:
origin/master:backend/main.py:75:# CORS for the web UI. allow_origins='' with
allow_credentials=True is rejected by browsers per the
origin/master:backend/main.py:79:app.add_middleware(
origin/master:backend/main.py:80:    CORSMiddleware,
origin/master:backend/main.py:81:    allow_origins=_cors_origins_from_env(),
origin/master:backend/main.py:82:    allow_credentials=False,
origin/master:backend/main.py:108:app.add_middleware(
```

8. assistant (2026-06-21T19:13:09.911Z)

Part (a) is confirmed: ``RALLY_CORS_ALLOW_ORIGINS`` is read at import (``backend/main.py:65-72``) and applied to ``CORSMiddleware`` with ``allow_credentials=False`` (``backend/main.py:79-84``). FastAPI's ``CORSMiddleware`` answers the OPTIONS preflight automatically.

Now let me get the exact line numbers and verify the preflight handling, then dig into the config.json discovery path (the gotcha ? PR #293 RALLY_APP_HOME).

9. user (2026-06-21T19:13:12.063Z)

```
32:from fastapi.middleware.cors import CORSMiddleware
65:def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
75:# CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by browsers
per the
79:app.add_middleware(
80:    CORSMiddleware,
81:    allow_origins=_cors_origins_from_env(),
82:    allow_credentials=False,
108:app.add_middleware(
```

10. user (2026-06-21T19:13:13.780Z)

```
origin/master:backend/config/loader.py:4:merged with optional file overrides (config.json).
origin/master:backend/config/loader.py:19:from backend.utils.app_home import default_config_path
origin/master:backend/config/loader.py:32:# Config path resolution now lives in
``backend.utils.app_home.default_config_path()`` ? it honours
origin/master:backend/config/loader.py:33:# ``RALLY_APP_HOME`` (PACKAGING_PLAN.md P0a) and
equals ``Path("config.json")`` (CWD-relative) when
origin/master:backend/config/loader.py:34:# the env is unset, exactly the pre-P0a value. (The
old module-level ``DEFAULT_CONFIG_PATH`` constant
origin/master:backend/config/loader.py:36:# from a later RALLY_APP_HOME.)
origin/master:backend/config/loader.py:41:    replace). Used ONLY for the profile-preset
precedence path so a config.json sub-key
origin/master:backend/config/loader.py:54:    ""The EXPLICITLY chosen deployment profile, env
winning over config.json. ``"" if none
origin/master:backend/config/loader.py:60:    asserting a preset that was never applied. A bad
value in config.json is left to surface as
origin/master:backend/config/loader.py:69:    "and falling back to config.json
deployment.profile.", ep, "", ".join(PROFILE_PRESETS),
origin/master:backend/config/loader.py:108:def load_config(path: str | Path | None = None) ->
AppConfig:
origin/master:backend/config/loader.py:118:    cfg_path = Path(path) if path is not None else
default_config_path()
origin/master:backend/config/loader.py:130:    # A config.json that is valid JSON but not an
OBJECT (e.g. ``[]``, ``42``, ``"x"`` , ``null``) would
origin/master:backend/config/loader.py:149:    # Precedence: built-in defaults < profile preset
< config.json (< env/--set, applied at
```

```

origin/master:backend/config/loader.py:151: # explicitly selected (DEPLOYMENT_PROFILE env or
config.json deployment.profile). With no
origin/master:backend/config/loader.py:157: # Only an unrecognised value from config.json
reaches here (env typos already fell back
origin/master:backend/config/loader.py:168: # Record the profile actually applied ? the
env may have chosen it over config.json,
origin/master:backend/config/loader.py:176: # An active profile with no explicit
compute_target (e.g. config.json blanked it
origin/master:backend/config/loader.py:183: "config.json overrides it to %r ?
honoring the explicit override.",
origin/master:backend/config/loader.py:203: """Write a fresh default config.json.
origin/master:backend/config/loader.py:208: cfg_path = Path(path) if path is not None else
default_config_path()
origin/master:backend/config/models.py:6:The models support graceful migration from old
config.json files via extra="allow".
origin/master:backend/config/models.py:241: (under `stitching.watermark` in config.json) to
turn it off. The text is fully
origin/master:backend/config/models.py:353: behaves exactly as before this section existed. A
profile is opt-in (`config.json`)
origin/master:backend/config/models.py:363: # Where the core runs. Set by the profile preset;
overridable in config.json. Inert in M1
origin/master:backend/config/models.py:410: # Allow extra keys during transition so existing
config.json files do not break.
origin/master:backend/config/presets.py:9: built-in model defaults < profile preset <
config.json < env < worker --set
origin/master:backend/config/presets.py:13:(`config.json` `deployment.profile` or the
`DEPLOYMENT_PROFILE` env var); with no
origin/master:backend/config/presets.py:31:# when config.json explicitly overrides
compute_target into a state inconsistent with the profile.
origin/master:backend/eval/served_gate_a.py:35:# Production windowing operating point
(config.json indexing defaults ? the values
origin/master:backend/eval/served_gate_a.py:44: config.json's `indexing.fusion` block, with
Gate A at `min_crossings` and the
origin/master:backend/eval/windowing.py:152: default in `config.json` ? the module-level
:data:`SERVED` is the *default* operating point;
origin/master:backend/main.py:16:# PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when
RALLY_APP_HOME is set (the packaged
origin/master:backend/main.py:20:from backend.utils.app_home import env_file_to_load,
resolve_output_dir
origin/master:backend/main.py:67: default `[ ]`. Read from the ENV (not a cwd
`config.json`) so importing `backend.main`
origin/master:backend/main.py:94: ENV (not config.json) so importing `backend.main` does
zero filesystem I/O; a routable/edge
origin/master:backend/main.py:253:def load_config(config_path: str = "config.json") -> Dict[str,
Any]:
origin/master:backend/main.py:254: cfg = load_app_config(config_path)
origin/master:backend/main.py:945: # P0a: a RELATIVE output dir roots under the app-home
(RALLY_APP_HOME); with the env
origin/master:backend/main.py:970: f"use a different --output-dir
(RALLY_APP_HOME) to run an independent instance.")
origin/master:backend/pipeline/detectors/stub_runner.py:39: """Settings for the CPU stub
runner (built from `config.json` -> `indexing.stub`)."""
origin/master:backend/pipeline/detectors/tracknet_runner.py:32: """Resolved settings for a
TrackNet WSL run (built from config.json -> indexing.tracknet)."""
origin/master:backend/pipeline/detectors/tracknet_runner.py:133: "in config.json
to the WSL path of your trained TrackNetV4 weights."
origin/master:backend/pipeline/detectors/wasb_infer.py:65: with
initialize(config_path="configs", version_base=None):
origin/master:backend/pipeline/detectors/wasb_runner.py:32: """Resolved settings for a WASB
WSL run (built from config.json -> indexing.wasb)."""
origin/master:backend/pipeline/validators/base.py:29: - config: Dict parsed from
config.json.
origin/master:backend/utils/app_home.py:3:PACKAGING_PLAN.md P0a. Today every piece of mutable
state ? `config.json`, `.env`,
origin/master:backend/utils/app_home.py:5:`scripts/doctor.py` writes `config.json` to the

```

```

repo root while the server reads it from
origin/master:backend/utils/app_home.py:10: * ``RALLY_APP_HOME`` env set ? that directory is
the app home (the packaged launcher sets it).
origin/master:backend/utils/app_home.py:11: * ``RALLY_APP_HOME`` unset ? ``Path(".")`` i.e.
the CWD, exactly the pre-P0a behaviour.
origin/master:backend/utils/app_home.py:14:is computed by :func:`default_os_app_home` but is
**NOT** auto-selected here ? the launcher (P2)
origin/master:backend/utils/app_home.py:15:calls it and exports ``RALLY_APP_HOME`` explicitly.
Keeping the resolver opt-in means importing
origin/master:backend/utils/app_home.py:30:APP_HOME_ENV = "RALLY_APP_HOME"
origin/master:backend/utils/app_home.py:32:#: OS-app-dir leaf used by
:func:`default_os_app_home` (the product brand).
origin/master:backend/utils/app_home.py:36:def app_home() -> Path:
origin/master:backend/utils/app_home.py:39:    ``RALLY_APP_HOME`` (``~`` expanded) when set;
otherwise ``Path(".")`` ? the CWD, byte-identical
origin/master:backend/utils/app_home.py:49:    """A path under :func:`app_home` (e.g.
``app_path("output")``)."""
origin/master:backend/utils/app_home.py:50:    return app_home().joinpath(*[str(p) for p in
parts])
origin/master:backend/utils/app_home.py:53:def default_config_path() -> Path:
origin/master:backend/utils/app_home.py:54:    """`<app-home>/config.json` ? the config file
the loader reads and ``--setup`` writes.

```

11. assistant (2026-06-21T19:13:15.979Z)

This is the crux. Let me read the `app\_home.py` resolver and the `loader.py` to determine the exact config.json resolution order.

12. user (2026-06-21T19:13:17.629Z)

"""Resolved application-data HOME for a packaged, batteries-included install.

PACKAGING_PLAN.md P0a. Today every piece of mutable state ? ``config.json``, ``.env``, the SQLite DB, ``output/`` ? is resolved RELATIVE TO THE CURRENT WORKING DIRECTORY, and ``scripts/doctor.py`` writes ``config.json`` to the repo root while the server reads it from the CWD. Launched from an OS app menu (CWD ? repo root) those disagree and the user silently "loses" their index. This module introduces ONE resolver every state path roots on.

Design contract ? **DEFAULT IS BYTE-IDENTICAL TO TODAY**:

```

* ``RALLY_APP_HOME`` env set ? that directory is the app home (the packaged launcher sets
it).
* ``RALLY_APP_HOME`` unset ? ``Path(".")`` i.e. the CWD, exactly the pre-P0a behaviour.

```

The OS-default location (``%APPDATA%`` / ``~/Library/Application Support`` / ``~/.local/share``) is computed by :func:`default\_os\_app\_home` but is **NOT** auto-selected here ? the launcher (P2) calls it and exports ``RALLY\_APP\_HOME`` explicitly. Keeping the resolver opt-in means importing this module has zero behavioural effect on the repo/dev/CI/test paths (no surprise writes into a real user profile during a test run).

stdlib-only (os, platform, pathlib) so it is safe to import at ``backend.main`` line 1 ? before the heavier pydantic-backed ``backend.config`` package ? keeping the early ``load\_dotenv`` path light.

```

from __future__ import annotations

```

```

import os
import platform
from pathlib import Path

```

```

#: Environment variable the packaged launcher sets to pin the app-data home.
APP_HOME_ENV = "RALLY_APP_HOME"

```

```

#: OS-app-dir leaf used by :func:`default_os_app_home` (the product brand).
APP_DIR_NAME = "Khelsutra"

```

```

def app_home() -> Path:
    """The resolved application-data home.

    ``RALLY_APP_HOME`` (``~`` expanded) when set; otherwise ``Path(".")`` ? the CWD, byte-
    identical
    to the pre-P0a behaviour. Does NOT create the directory (callers create on write).
    """
    env = os.environ.get(APP_HOME_ENV)
    if env and env.strip():
        return Path(env.strip()).expanduser()
    return Path(".")

def app_path(*parts: str | os.PathLike[str]) -> Path:
    """A path under :func:`app_home` (e.g. ``app_path("output")``)."""
    return app_home().joinpath(*[str(p) for p in parts])

def default_config_path() -> Path:
    """``<app-home>/config.json`` ? the config file the loader reads and ``--setup`` writes.

    With ``RALLY_APP_HOME`` unset this is ``Path("config.json")`` (CWD-relative), unchanged.
    """
    return app_home() / "config.json"

def default_env_path() -> Path:
    """``<app-home>/.env`` ? the dotenv file the server prefers (e.g. ``GEMINI_API_KEY``)."""
    return app_home() / ".env"

def env_file_to_load() -> Path | None:
    """The ``.env`` the server should load, or ``None`` meaning "use python-dotenv's DEFAULT
    (frame-relative walk-up) search" ? exactly the pre-P0a bare ``load_dotenv()`` behaviour.

    Returns the app-home ``.env`` ONLY when ``RALLY_APP_HOME`` is set AND that file exists. With
    the
    env UNSET this returns ``None`` so the caller keeps the historical search and does NOT newly
    prefer a stray CWD ``.env`` (bare ``load_dotenv()`` walks up from the caller file, not the
    CWD).
    """
    home = app_home()
    if str(home) == ".": # env unset (or pinned to CWD) ? historical load_dotenv() search
        return None
    candidate = home / ".env"
    return candidate if candidate.exists() else None

def resolve_output_dir(output_dir: str) -> str:
    """Root a RELATIVE ``output_dir`` under :func:`app_home`; pass an ABSOLUTE one through.

    With ``RALLY_APP_HOME`` UNSET the input is returned VERBATIM (the default ``"output"`` when
    empty) ? byte-identical to the pre-P0a code that stored ``--output-dir`` as-is (no pathlib
    normalisation of ``./x`` / separators / trailing slash). With it SET, a relative dir like
    ``"output"`` becomes ``<app-home>/output`` so the SQLite DB and reels live with the rest of
    the
    app's state; an absolute path ? or a Windows drive-anchored one (``C:\\x`` / ``C:x``) ?
    passes
    through untouched.
    """
    home = app_home()
    if str(home) == ".": # env unset (or pinned to CWD) ? verbatim, today's behaviour
        return output_dir or "output"
    if not output_dir:
        return str(home / "output")

```

```

if os.path.isabs(output_dir) or os.path.splitdrive(output_dir)[0]:
    return output_dir
return str(home / output_dir)

def default_os_app_home(app_name: str = APP_DIR_NAME) -> Path:
    """The OS-conventional per-user app-data dir (for the packaged LAUNCHER to export as
    ``RALLY_APP_HOME`` ? NOT auto-selected by :func:`app_home`).

    * Windows ? ``%APPDATA%\\<app_name>`` (``~/AppData/Roaming/<app_name>`` if unset)
    * macOS ? ``~/Library/Application Support/<app_name>``
    * Linux/other ? ``$XDG_DATA_HOME/<app_name>`` (``~/.local/share/<app_name>`` if unset)
    """
    system = platform.system()
    if system == "Windows":
        base = os.environ.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
    elif system == "Darwin":
        base = str(Path.home() / "Library" / "Application Support")
    else:
        base = os.environ.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
    return Path(base) / app_name

```

13. user (2026-06-21T19:13:18.427Z)

```

def _get_builtin_defaults() -> dict[str, Any]:
    """Return a plain dict of the current built-in defaults.

    The AppConfig model is the single source of truth for defaults.
    """
    return AppConfig().model_dump(mode="json")

def load_config(path: str | Path | None = None) -> AppConfig:
    """Load configuration with the following precedence:

    1. Built-in defaults defined in the Pydantic models.
    2. Values from the JSON file (if present and readable).
    3. (Future) environment / CLI overrides.

    Missing file or unreadable file ? returns a fully defaulted AppConfig.
    Unknown keys in the file are tolerated (extra="allow" on the model).
    """
    cfg_path = Path(path) if path is not None else default_config_path()

    file_data: dict[str, Any] = {}
    if cfg_path.exists():
        try:
            with open(cfg_path, "r", encoding="utf-8") as f:
                file_data = json.load(f)
        except Exception as exc:
            # Non-fatal: continue with defaults + whatever we could parse.
            # In production we might log here.
            logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial data.")

    # A config.json that is valid JSON but not an OBJECT (e.g. `[ ]`, `42`, `"x"`, `null`) would
    # otherwise crash startup: a truthy non-mapping hits `.items()` in _unknown_key_paths, and
    any
    # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults + warn, per
    the
    # loader's "bad input is non-fatal" contract.
    if not isinstance(file_data, dict):
        logger.warning("%s is not a JSON object (got %s); ignoring it and using defaults.",

```

14. assistant (2026-06-21T19:13:22.656Z)

Now I understand the config.json resolution precisely. But there's a critical wrinkle ?
`backend/main.py:253` defines its own `load\_config(config\_path: str = "config.json")`. Let me check whether the server's startup path uses the env-aware `default\_config\_path()` or hardcodes `config.json` (CWD-relative). This determines whether RALLY_APP_HOME actually governs config discovery at server boot.

15. user (2026-06-21T19:13:24.325Z)

```
    _schedule_next_drain_if_waiting,  
)
```

Legacy thin wrapper for any remaining direct calls during transition.

New code should import load_app_config / AppConfig from backend.config directly.

```
def load_config(config_path: str = "config.json") -> Dict[str, Any]:  
    cfg = load_app_config(config_path)  
    return cfg.model_dump(mode="json")
```

--- API Models ---

```
class ProcessRequest(BaseModel):  
    video_path: str  
    player_pool: Optional[List[str]] = None  
    max_frames: Optional[int] = None  
    segmenter: Optional[str] = None
```

16. user (2026-06-21T19:13:25.797Z)

```
50:     load_config as load_app_config,  
53:     enforce_startup_checks,  
67:     default "[""]`. Read from the ENV (not a cwd `config.json`) so importing  
    `backend.main`  
94:     ENV (not config.json) so importing `backend.main` does zero filesystem I/O; a  
    routable/edge  
252:# New code should import load_app_config / AppConfig from backend.config directly.  
253:def load_config(config_path: str = "config.json") -> Dict[str, Any]:  
254:     cfg = load_app_config(config_path)  
779:     cfg = load_config() # legacy wrapper still works, returns dict for now  
854:         enforce_startup_checks(config, logger=logger, include_exposure=True)  
860:def main():  
920:     stitching = load_app_config().stitching  
939:     global_config = load_app_config() # returns typed AppConfig
```

17. assistant (2026-06-21T19:13:31.240Z)

There's an important subtlety. Line 779 calls `load\_config()` (the legacy wrapper) with NO argument, so `config\_path` defaults to `config.json` (CWD-relative, ignoring RALLY_APP_HOME). But lines 854/920/939 call `load\_app\_config()` with no argument, which uses `default\_config\_path()` (RALLY_APP_HOME-aware). Let me check the server boot path (`main()`) and the startup checks) to determine which one governs the actual exposed deploy, and confirm line 779's context.

18. user (2026-06-21T19:13:32.975Z)

```
        raise HTTPException(status_code=500, detail=f"Highlight stitching failed: {str(e)}")  
    from e
```

--- CLI Debug Utility & Orchestration ---

```
def run_cli_diagnose_clip(video_path: str, timestamp: float):  
    """CLI debugging utility to examine segment properties around a timestamp."""  
    print("=" * 60)  
    print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")  
    print("=" * 60)
```

```

cfg = load_config() # legacy wrapper still works, returns dict for now

# 1. Validation check diagnostics
print("[DIAGNOSTIC] Running validation framework...")
results = registry.run_all(video_path, cfg)
for r in results:
    status_symbol = "[PASS]" if r.passed else "[FAIL]"
    print(f" {status_symbol} {r.validator_name}: {r.message}")
    if r.details:
        print(f"      Details: {r.details}")

# 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)

```

19. user (2026-06-21T19:13:33.478Z)

```

print(" [ERROR] No visual frames were extracted in the sample range.")

print("=" * 60)

```

```

def _enforce_startup_or_exit(config) -> None:
    """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL finding
    BEFORE
    any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only storage) OR,
    for
    the exposed server (``include_exposure=True``, M9 hardening), a half-configured exposure
    (edge
    auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when no
    ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU probe
    here ?
    the server stays torch-light; the detector healthcheck is the GPU authority. Extracted so
    the
    fail-fast wiring + the default-OFF no-op are unit-testable."""
    try:
        enforce_startup_checks(config, logger=logger, include_exposure=True)
    except StartupCheckError as e:
        logger.error("[STARTUP] refusing to start: %s", e)
        sys.exit(1)

def main():
    parser = argparse.ArgumentParser(description="Sports Highlight Indexer Orchestrator")
    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
    parser.add_argument("--output-dir", default="output", help="Directory where processed
    clips/highlights are saved")
    parser.add_argument("--setup", action="store_true", help="Launch dependency checker and
    setup utility")
    parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a video clip
    to run detailed diagnostics on")
    parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
    server")
    parser.add_argument("--app", action="store_true",
        help="Launch the app: start the local server (localhost-only) and open
    the "
        "bundled UI in your browser. Falls back to a free port if --port is
    busy.")
    parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI server
    on")
    parser.add_argument("--host", default=None,
        help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0 when "
        "security.edge_auth_enabled. Expose on a network ONLY behind an
    auth gate.")
    parser.add_argument("--max-frames", type=int, help="Limit processing to the first N sub-
    sampled frames for debugging")

```

```

    available_segmenters = list(segmenter_registry.list_available().keys())
    parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
    parser.add_argument("--trim-start", type=float, help="Start time in seconds for the debug
trim segment feature")
    parser.add_argument("--trim-end", type=float, help="End time in seconds for the debug trim
segment feature")
    parser.add_argument("--trim-output", help="Output filename for the debug trimmed segment
(saves in output-dir unless absolute path)")

    args = parser.parse_args()

    if args.setup:
        # Invoke the diagnostics/bootstrap script (renamed from the old root
        # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
        # build system). Resolve its path from this file so `indexer --setup`
        # works regardless of the caller's cwd.
        import subprocess
        doctor_path = os.path.join(
            os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
            "scripts", "doctor.py",
        )
        print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
        subprocess.run([sys.executable, doctor_path])
        return

    if args.trim_start is not None and args.trim_end is not None:
        if not args.video_path:
            print("[ERROR] Please provide --video-path to extract a segment.")
            return

        # Determine output path
        out_filename = args.trim_output or "trimmed_segment.mp4"
        if os.path.isabs(out_filename):
            out_path = out_filename
            out_dir = os.path.dirname(out_path)
            out_name = os.path.basename(out_path)
        else:
            out_dir = resolve_output_dir(args.output_dir) # P0a: app-home aware (CWD-identical
when unset)
            out_path = os.path.join(out_dir, out_filename)
            out_name = out_filename

        os.makedirs(out_dir, exist_ok=True)
        print(f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s from
{args.video_path}...")

        # global_config isn't initialized this early; load the typed config so the trim
        # clip carries the same configured stitching paddings + watermark as a real
        # highlight (watermark default-on).
        stitching = load_app_config().stitching
        compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
                                end_padding=stitching.end_padding,
watermark=stitching.watermark)
        try:
            res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
            print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
        except Exception as e:
            print(f"[ERROR] Failed to compile trimmed segment: {e}")
        return

    if args.debug_clip is not None:
        if not args.video_path:
            print("[ERROR] Please provide --video-path to debug a specific clip.")

```

```

        return
    run_cli_diagnose_clip(args.video_path, args.debug_clip)
    return

# Initialize Global Config and DB
global global_config, db_instance
global_config = load_app_config() # returns typed AppConfig
_enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE profile

# The CLI --output-dir overrides the configured output directory. It now lives
# on the typed model (OutputConfig.output_dir), so every consumer ? including
# /api/compile ? reads the same value instead of a write-only runtime hack.
# P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env

```

20. assistant (2026-06-21T19:13:39.711Z)

This confirms the server boot path. At line 939, `main()` calls `load_app_config()` with NO argument ? resolves via `default_config_path()` ? which honors `RALLY_APP_HOME`. Then `_enforce_startup_or_exit` runs the exposure checks on THAT config. So the exposed server reads `config.json` from `<RALLY_APP_HOME>/config.json` when the env is set, or `./config.json` (CWD) when unset.

Now let me confirm two final things: (1) `_cors_origins_from_env` reads `os.environ` at import time (module-level `add_middleware` call at line 79, before any config load), and (2) verify the `--server/--app` path also goes through `main()` ? `load_app_config()`. Let me also check the preflight test to confirm `OPTIONS` is answered, and confirm `doctor.py --setup` writes config to the app-home (the place the owner would "create/edit").

21. user (2026-06-21T19:13:41.882Z)

```

below, which asserts a non-NULL owner_id is actually persisted.)"""
client, db, _m = _client(tmp_path, monkeypatch, AppConfig()) # edge_auth_enabled=False
v = tmp_path / "x.mp4"
v.write_bytes(b"x")
r = client.post("/api/process", json={"video_path": str(v)})
assert r.status_code == 202
assert db.get_job(r.json()["job_id"])["owner_id"] is None

def test_cors_origins_from_env():
    """CORS origins come from RALLY_CORS_ALLOW_ORIGINS (import-safe, no cwd config I/O); default
    *."""
    import backend.main as m
    assert m._cors_origins_from_env({}) == ["*"]
    assert m._cors_origins_from_env({"RALLY_CORS_ALLOW_ORIGINS": ""}) == ["*"]
    assert m._cors_origins_from_env(
        {"RALLY_CORS_ALLOW_ORIGINS": "https://a.example, https://b.example"}
    ) == ["https://a.example", "https://b.example"]

def test_process_edge_auth_missing_token_401(tmp_path, monkeypatch):
    client, _db, _m = _client(tmp_path, monkeypatch, _cfg(True))
    v = tmp_path / "x.mp4"
    v.write_bytes(b"x")
    r = client.post("/api/process", json={"video_path": str(v)}) # no Cf-Access header
    assert r.status_code == 401

def test_process_edge_auth_valid_token_sets_owner(tmp_path, monkeypatch, keypair):
    key, jwks = keypair
    client, db, m = _client(tmp_path, monkeypatch, _cfg(True))
    monkeypatch.setattr(m.edge_auth, "_default_jwks_fetcher", lambda td: jwks)
    v = tmp_path / "x.mp4"
    v.write_bytes(b"x")
    r = client.post("/api/process", json={"video_path": str(v)},

```

```

        headers={edge_auth.CF_ACCESS_HEADER: _token(key)})
assert r.status_code == 202
assert db.get_job(r.json()["job_id"])["owner_id"] == "u@example.com"

```

22. user (2026-06-21T19:13:43.078Z)

```

origin/master:scripts/doctor.py:7:# location, cwd-independent) is used for the repo-bundled
requirements.txt. NOTE: config.json is
origin/master:scripts/doctor.py:8:# NO LONGER written here ? it follows
backend.utils.app_home.default_config_path() (RALLY_APP_HOME,
origin/master:scripts/doctor.py:9:# else CWD), the SAME path the server READS, so --setup and
the server can't disagree (P0a).
origin/master:scripts/doctor.py:46:     """Initialize a default config.json if it doesn't exist.
origin/master:scripts/doctor.py:52:         from backend.config import save_default_config
origin/master:scripts/doctor.py:53:         from backend.utils.app_home import
default_config_path
origin/master:scripts/doctor.py:60:     # (RALLY_APP_HOME, else CWD/config.json) ? previously
this wrote to the repo root while
origin/master:scripts/doctor.py:61:     # the server read from the CWD, so --setup and the server
disagreed once CWD ? repo root.
origin/master:scripts/doctor.py:62:     config_path = str(default_config_path())
origin/master:scripts/doctor.py:68:         saved_path = save_default_config(config_path)
origin/master:scripts/doctor.py:69:         print(f"[SUCCESS] config.json initialized with
default settings at {saved_path}.")
origin/master:scripts/doctor.py:71:         print(f"[ERROR] Failed to create config.json via
loader: {e}")

```

23. assistant (2026-06-21T19:13:48.009Z)

This fully resolves the gotcha. The config.json discovery is now consistent: both `--setup` (doctor.py) and the server READ/WRITE config via `default\_config\_path()` (RALLY_APP_HOME, else CWD). They cannot disagree by design (P0a fixed exactly the bug where doctor wrote to repo root while server read from CWD).

Let me confirm the `--server` flag also routes through `main()` and that there's no separate uvicorn entry that bypasses it, plus verify the module-level CORS middleware reads env at import (before config load). Let me check the server launch.

24. user (2026-06-21T19:13:49.843Z)

```

# P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
# unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to before.
global_config.output.output_dir = resolve_output_dir(args.output_dir)
os.makedirs(global_config.output.output_dir, exist_ok=True)

db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)

# P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job sweep
below.
# The job worker is in-process, so a concurrent process opening this DB would sweep THIS
one's
# in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-released on
exit
# or crash) is keyed to db_path, so a different --output-dir/DB still runs concurrently.
global _instance_lock
try:
    _instance_lock = acquire_lock(db_path + ".lock")
except OSError as exc:
    # The lock path isn't writable (read-only dir / perms). Don't crash and don't falsely
claim a
    # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure must not
block a
    # legitimate single instance).
    logger.warning("[JOBS] Could not create the single-instance lock at %s.lock (%s);

```

```

proceeding "
    "WITHOUT the second-instance guard.", db_path, exc)
    _instance_lock = None
else:
    if _instance_lock is None:
        print(f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
            f"start ? a second instance would corrupt in-flight job state. Stop the other
one, or "
            f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance.")
        sys.exit(1)

db_instance = Database(db_path)

# Crash recovery (#16): jobs left queued/running by a previous process are dead ?
# the executor is in-process. Mark them failed so pollers see a terminal state.
swept = db_instance.fail_stale_jobs()
if swept:
    logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed.")

# CLI processing mode (no web UI needed). --app/--server always run the server, never CLI
single-shot.
if args.video_path and not args.server and not args.app:
    print(f"[CLI] Processing video file: {args.video_path}")
    # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is fetched to
scratch.
    # The local-existence fast-fail only applies to a local input (stat the RESOLVED key,
not the
    # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error, not a
crash.
    storage_cfg = getattr(global_config, "storage", None)
    try:
        input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
    except ValueError as e:
        print(f"[FAIL] {e}")
        return
    if input_ref.backend == "local" and not os.path.exists(input_ref.key):
        print(f"[FAIL] Video file not found: {args.video_path}")
        return
    local_video = storage.acquire_local_input(args.video_path, storage_cfg)

    video_id = compute_video_id(local_video)
    was_reingest = db_instance.get_video(video_id) is not None

    # Validate (with-context variant surfaces ffprobe_meta for the report)
    print("[CLI] Validating...")
    val_results, val_context = registry.run_all_with_context(local_video, global_config)
    failures = [r for r in val_results if not r.passed]

    # Save video status
    status = "failed" if failures else "processed"
    db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r in
val_results])

    seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
    segmenter_actual = None
    segmentation_ran = False
    try:
        print("\n" + "="*40)
        print("VALIDATION SUMMARY")
        print("="*40)
        for r in val_results:
            status_symbol = "[PASS]" if r.passed else "[FAIL]"

```

```

        print(f"{status_symbol} {r.validator_name}: {r.message}")
    print("="*40 + "\n")

    if failures:
        print("[FAIL] Video failed checks. Indexing halted.")
        return

    # Index
    segmenter_cls = segmenter_registry.get(seg_name)
    if not segmenter_cls:
        print(f"[FAIL] Segmenter '{seg_name}' not found.")
        return

```

25. assistant (2026-06-21T19:13:52.970Z)

I have everything I need. Let me confirm the final piece ? that `--server`/`--app` launches uvicorn via `main()` (so the config-load + startup-check path is exercised before serving), and confirm the CORS middleware is wired at module import (line 79), independent of any config load.

26. user (2026-06-21T19:13:55.160Z)

```

982:     if args.video_path and not args.server and not args.app:
1063:         if args.app or args.server or not args.video_path:
1064:             if args.app:
1092:                 uvicorn.run(app, host=bind_host, port=port)
1094: if __name__ == "__main__":

```

27. assistant (2026-06-21T19:14:10.728Z)

Confirmed. The full chain for the exposed server is: `main()` (line 860) ? `load\_app\_config()` with no arg (line 939, RALLY_APP_HOME-aware via `default\_config\_path()`) ? `\_enforce\_startup\_or\_exit` (line 940) ? `uvicorn.run(app, ...)` (line 1092). The `app` object already has CORSMiddleware wired at module import (line 79) with `allow\_origins=\_cors\_origins\_from\_env()` reading `os.environ` and `allow\_credentials=False`.

All verification is complete. Let me record findings.

The CORS claim is fully confirmed. The "create/edit config.json on the box" instruction is technically correct but underspecified ? there's a real owner-action gotcha around RALLY_APP_HOME.

28. user (2026-06-21T19:14:31.025Z)

<tool_use_error>InputValidationError: StructuredOutput was called with input that could not be parsed as JSON.
 You sent (first 200 of 4187 bytes): {"id": "deploy-alpha-cors-config", "claim": "Engine locks CORS to the SPA origin via env RALLY_CORS_ALLOW_ORIGINS (no code change), with allow_credentials=false, and answers the cross-origin OPTIONS p
 Common causes: unescaped backslashes in file paths (use / or \\), unescaped control characters, or truncated output. Retry with valid JSON.</tool_use_error>

29. assistant (2026-06-21T19:14:33.377Z)

I had a malformed JSON (the evidence array was broken). Let me call it correctly with a proper structured object.

30. user (2026-06-21T19:14:54.321Z)

Structured output provided successfully

31. assistant (2026-06-21T19:15:07.495Z)

Verification complete. The claim is confirmed for origin/master, with one minor doc-clarity gap on config.json location.

Summary:

Part (a) ? fully confirmed:

- ``RALLY_CORS_ALLOW_ORIGINS`` is read from ``os.environ`` at module import (``backend/main.py:65-72``) and applied to ``CORSMiddleware`` with ``allow_credentials=False`` (``backend/main.py:79-84``). No engine code change needed ? it is env-driven and the middleware is fixed at startup.
- FastAPI's ``CORSMiddleware`` answers the cross-origin OPTIONS preflight automatically; the engine adds no custom handler. The env-parsing contract is locked by ``tests/test_edge_auth.py:205-211``.

Part (b) ? the gotcha (config.json discovery): PR #293's ``RALLY_APP_HOME`` does govern config.json. Resolution order on origin/master:

1. Explicit path argument (not used on the server boot path).
2. ``$RALLY_APP_HOME/config.json`` when ``RALLY_APP_HOME`` is set.
3. ``./config.json`` relative to the process CWD when unset (not the repo root).

The server boot path (``main()`` at ``backend/main.py:939``) calls ``load_app_config()`` with no argument, so it uses the env-aware ``default_config_path()`` (``backend/utils/app_home.py:53-56`` ? ``backend/config/loader.py:118``), and the M9 exposure startup checks run against that same config. Critically, ``scripts/doctor.py --setup`` writes config.json to the identical resolved path (``scripts/doctor.py:52-69``), so ``--setup`` and the server cannot disagree by construction ? P0a fixed exactly the older bug where doctor wrote to repo-root while the server read CWD.

The doc gap: §2.2 says "create/edit config.json on the box" without naming the location. The owner CAN put it somewhere the engine won't read ? e.g. editing ``./config.json`` but launching with a different CWD or with ``RALLY_APP_HOME`` exported. Because a missing file degrades silently to defaults (``edge_auth_enabled=False``), the security/jail/AUD block could be ignored without an obvious error. CORS itself is immune (env-driven), so only the in-config.json security block is at risk. Severity: minor. The owner-action note records the exact safe recipe and the fail-fast cross-check (PR #294) to confirm the right config.json was actually read.

Relevant files (engine origin/master): ``backend/main.py``, ``backend/utils/app_home.py``, ``backend/config/loader.py``, ``scripts/doctor.py``, ``tests/test_edge_auth.py``. Doc: ``C:\Users\avidu\Projects\khehsutra\deploy\DEPLOY_ALPHA.md`` (§2.2, §4.5).